ESTRUTURA DE DADOS

CURSO DE ENGENHARIA DA COMPUTAÇÃO

AULA DE HOJE

ORDENAÇÃO II

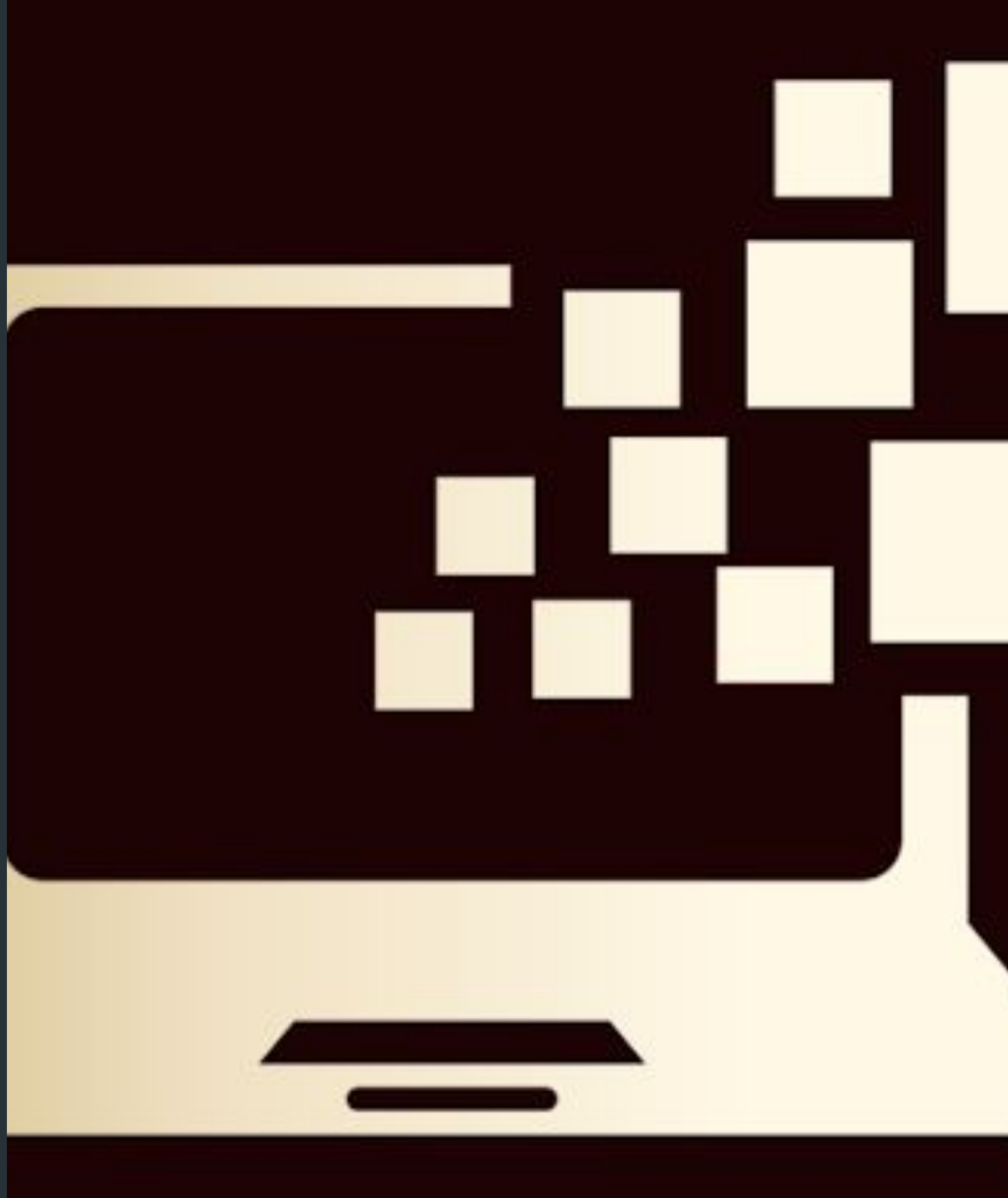
Introdução

- Os três métodos de ordenação analisados (insertionSort, selectionSort e bubbleSort) não são muito eficientes – $O(n^2)$.

- Pode-se esperar algum nível melhor de eficiência para um algoritmo de ordenação?

  - O limite $O(n^2)$ precisa ser quebrado para melhorar a eficiência e o tempo de execução.

  - Como isso pode ser feito?



Shellsort - Método dos Incrementos Descendentes:

- Proposto por Donald L. Shell em 1959
- Explora o fato de que o método de inserção direta apresenta desempenho aceitável quando o número de chaves é pequeno e/ou estas já estão parcialmente ordenadas.
- Consiste em dividir o vetor em H segmentos, de tal forma que cada um possua aproximadamente N/H chaves e classificarmos cada segmento separadamente.

Shellsort - Algoritmo

- Para implementar o método, o vetor V[1..N] é dividido em segmentos:
  - Segmento 1: V[1], V[H+1], V[2H+1], ...
  - Segmento 2: V[2], V[H+2], V[2H+2], ...
  - Segmento 3: V[3], V[H+3], V[2H+3], ...
  - ...
  - Segmento H: V[H], V[H+H], V[2H+H], ...
- Onde H é o incremento.
- Mas qual seria um valor adequado para H?

Shellsort - Algoritmo

- Extensivos estudos sugerem escolher incrementos que satisfaçam às seguintes condições:
  - $h_1 = 1$
  - $h_{i+1} = 3h_i + 1$, ... ,
  - h_t, para o qual h_{t+2} >= N
- Para n = 10000, tem-se a seqüência, 1,4,13,40,121,364,1093,3280
- Por simplificação, utilizaremos incrementos iguais a potências inteiras de 2

Aplicações de Ordenação

1

Num primeiro passo, para um certo H inicial, os segmentos formados são classificados por inserção direta;

2

Num segundo passo, o incremento H é diminuído (a metade do valor anterior), dando origem a novos segmentos, os quais também serão classificados por inserção direta;

3

O processo se repete até que H = 1. Quando for feita a classificação com H = 1, o vetor estará classificado.

Shellsort - Exemplo

Vetor inicial: (N = 16)

17 25 49 12 18 23 45 38 53 42 27 13 11 28 10 14

✓**Primeiro passo: H = 4**

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	⇦ Índice
17	25	49	12	18	23	45	38	53	42	27	13	11	28	10	14	
1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	⇦Segmento

✓ Quatro segmentos antes da ordenação (inserção direta):

11 17 18 53	23 25 28 42	10 27 45 49	12 13 14 38

seg. 1 seg. 2 seg. 3 seg. 4

✓ Segmento original após H = 4:

11 23 10 12 17 25 27 13 18 28 45 14 53 42 49 38

Shellsort - Exemplo

✓ **Segundo passo: H = 2**

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	⇦ Índice
11	23	10	12	17	25	27	13	18	28	45	14	53	42	49	38	
1	2	1	2	1	2	1	2	1	2	1	2	1	2	1	2	⇦ Segmento

✓ Dois segmentos antes da ordenação:

11 10 17 27 18 45 53 49	23 12 25 13 28 14 42 38
seg. 1	seg. 2

✓ Dois segmentos após a ordenação:

10 11 17 18 27 45 49 53	12 13 14 23 25 28 38 42
seg. 1	seg. 2

Shellsort - Exemplo

✓ **Segundo passo: H = 2**

✓ Segmento original após H = 2:
10 12 11 13 17 14 18 23 27 25 45 28 49 38 53 42

✓ **Terceiro passo: H = 1**

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	⇦ Índice
10	12	11	13	17	14	18	23	27	25	45	28	49	38	53	42	
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	

⇦ Segmento

✓ **Vetor final:**
10 11 12 13 14 17 18 23 25 27 28 38 42 45 49 53

Shellsort - Exemplo

Shellsort – Comentários e Análise

- Como o primeiro incremento usado é grande, os segmentos individuais são pequenos e as ordenações com inserção direta são razoavelmente velozes.

- Cada ordenação parcial dos segmentos favorece o desempenho dos passos seguintes, uma vez que a inserção direta é acelerada quando o vetor já se encontra parcialmente ordenado.

Shellsort – Comentários e Análise

Embora a cada passo se use incrementos menores, e portanto, segmentos maiores, estes se encontram praticamente ordenados.

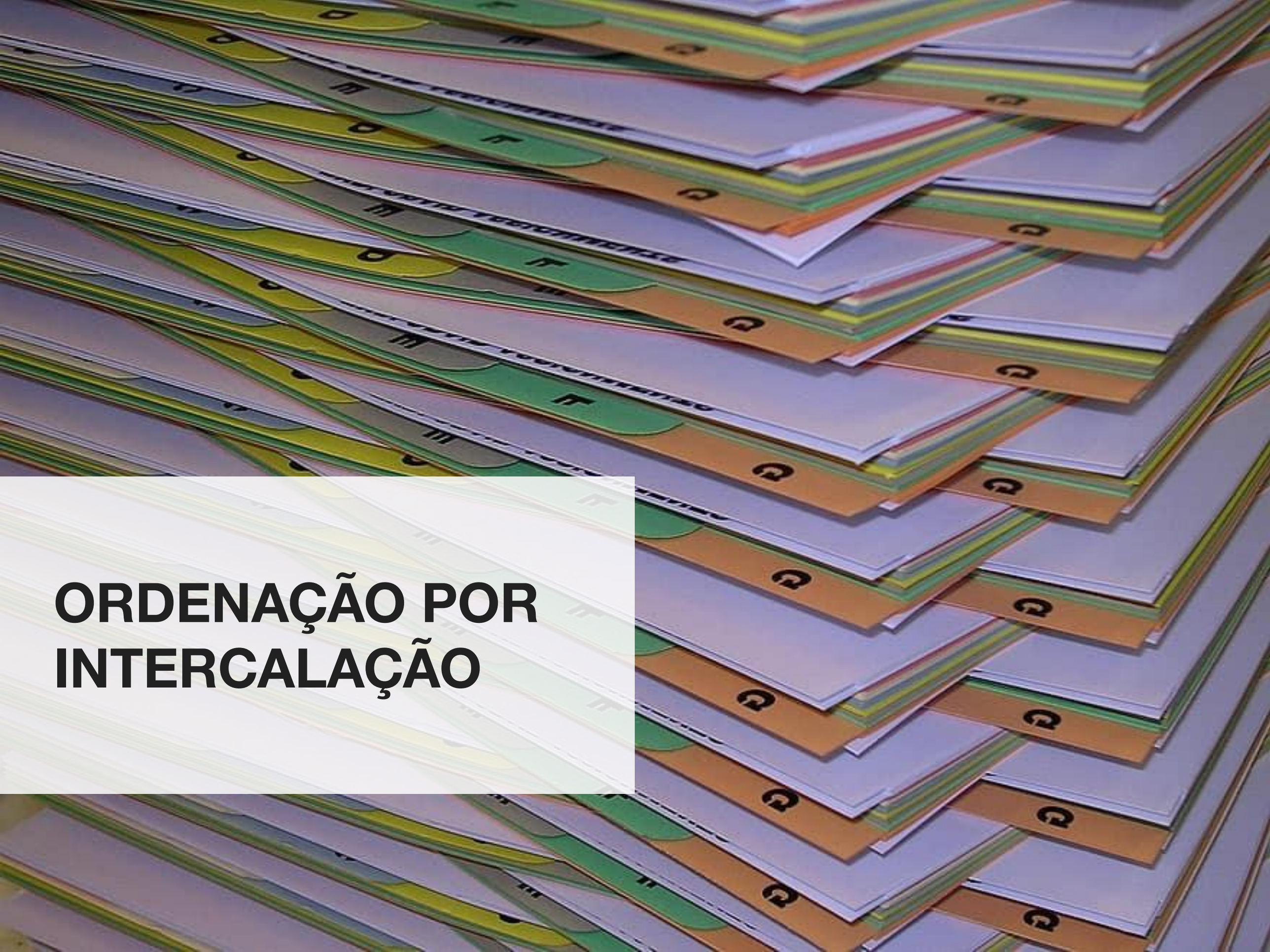
A análise de eficiência é da ordem de $O(n^{1,2})$ que ainda é melhor do que $O(n^2)$

Shellsort – Considerações Gerais

- Características que variam entre diferentes implementações:
  - Seqüência de incrementos;
  - Escolha do algoritmo de ordenação simples aplicado a cada segmento, exceto no último passo;
  - Escolha do algoritmo de ordenação simples aplicado ao último segmento, H = 1.

Shellsort – Considerações Gerais

- Características que variam entre diferentes implementações:
  - Por exemplo, Dobosiewicz usa ordenação por inserção para H != 1 e o método da bolha para H = 1.
  - Incerpi e Sedgewick usam o método da bolha para H != 1 e inserção direta para H = 1.
- Apesar de todas essas variações, todas essas versões têm desempenho melhor do que qualquer método de ordenação simples.



ORDENAÇÃO POR INTERCALAÇÃO

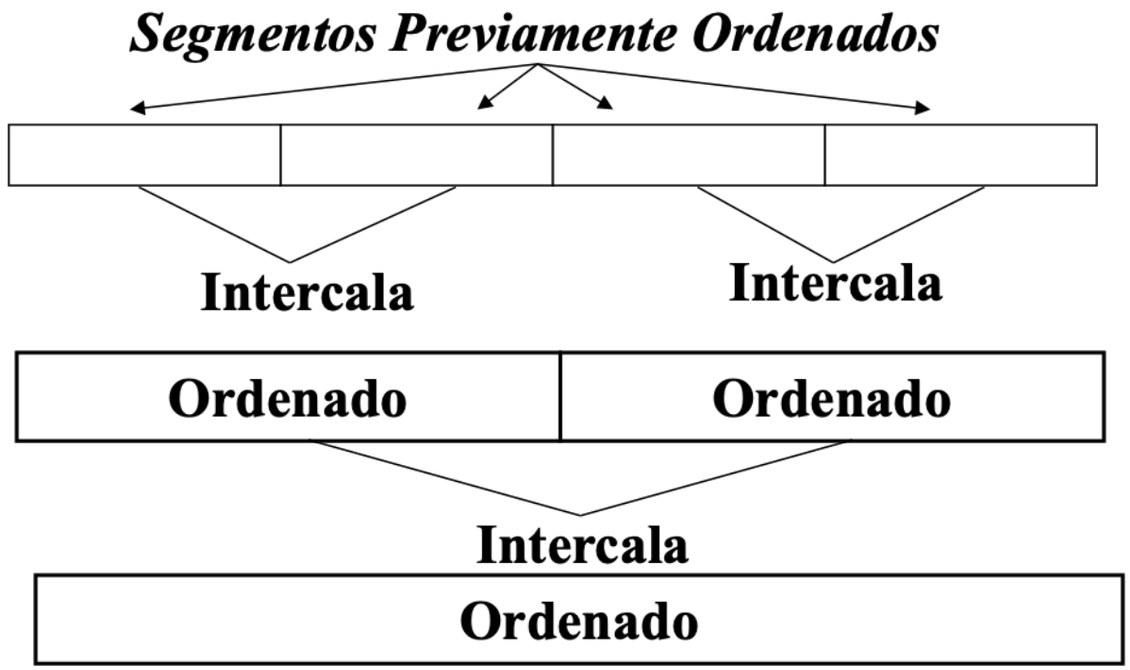
Classificação por Intercalação

- Este método consiste em dividir o vetor a ser classificado em dois ou mais segmentos, ordená-los separadamente e, depois, intercalá-los dois a dois, formando, cada par intercalado, novos segmentos ordenados, os quais serão intercalados entre si, repetindo-se o processo até que resulte apenas um único segmento ordenado.

- Para obter a ordenação dos segmentos iniciais, pode ser usado qualquer método de classificação

Classificação por Intercalação

- Esquema do Processo de Intercalação

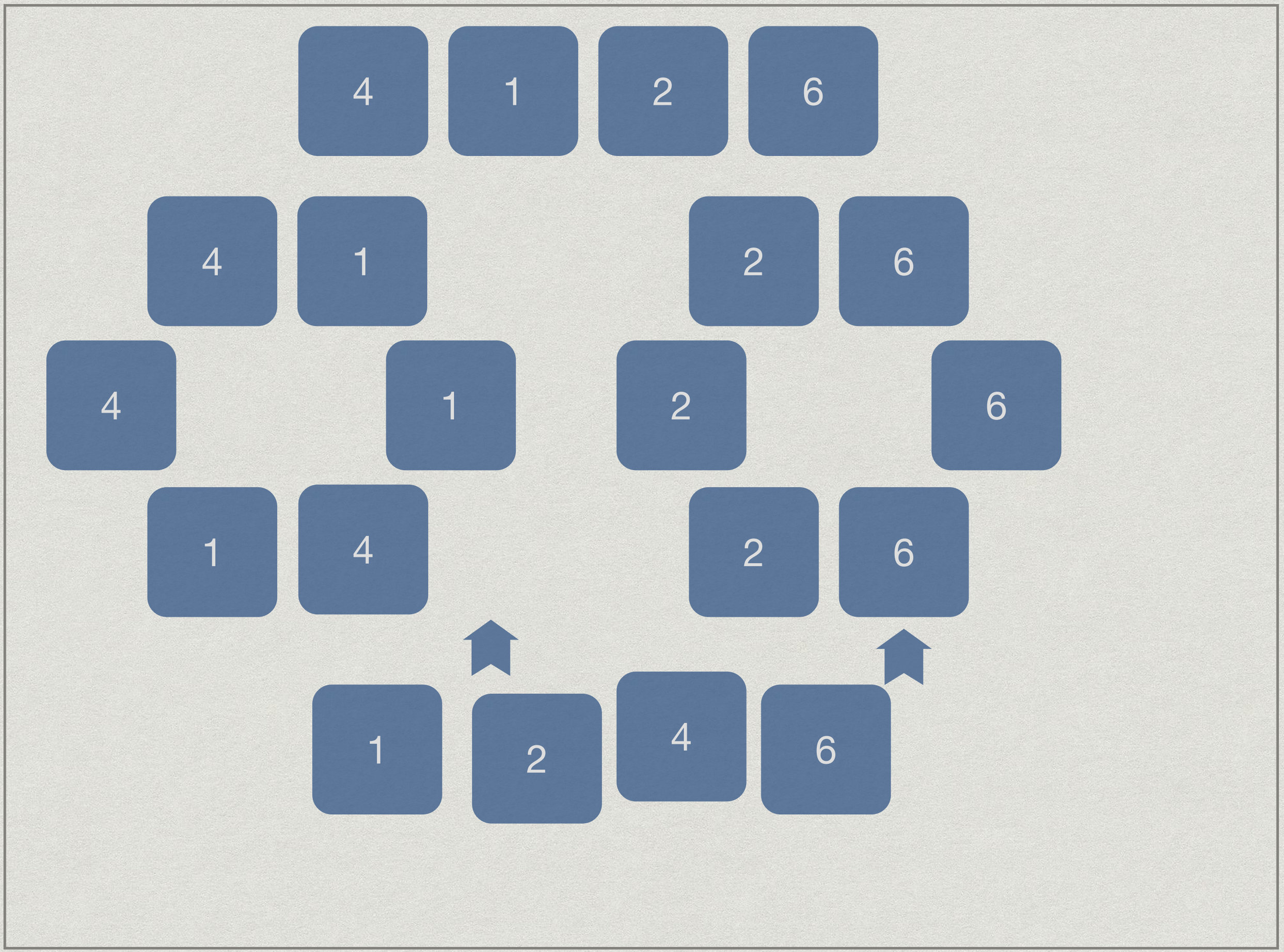


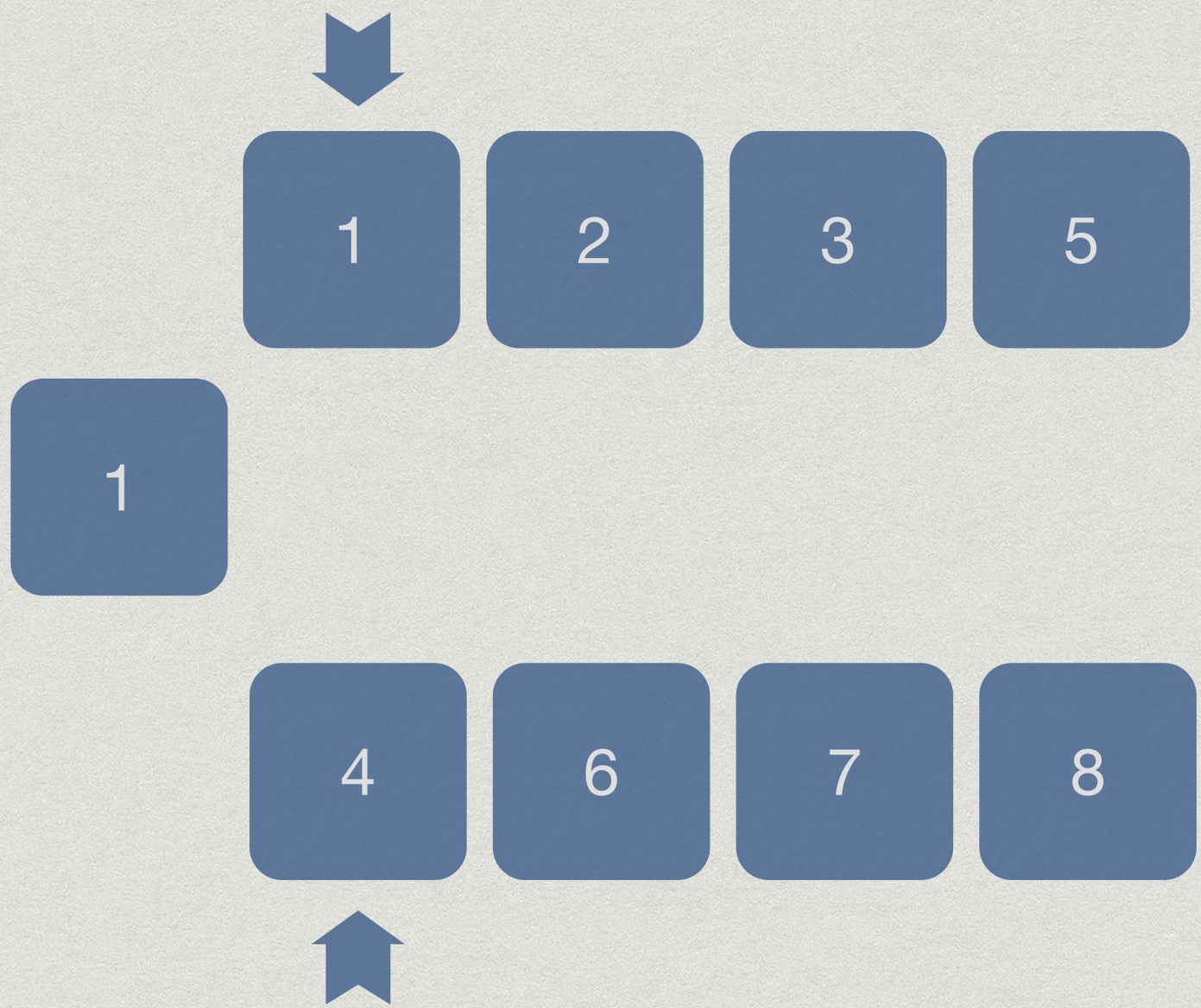
Método Mergesort

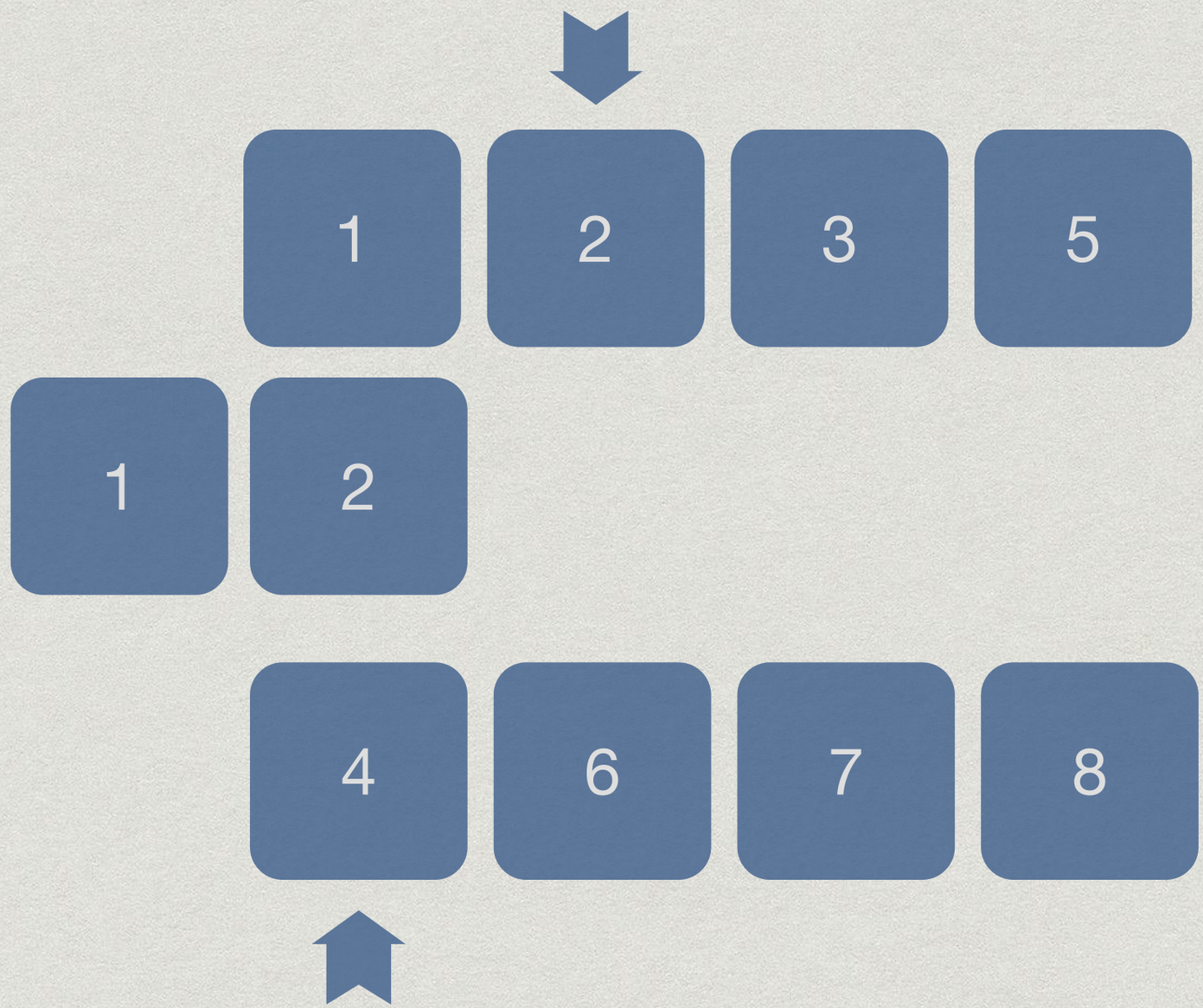
- Dividir recursivamente o vetor a ser ordenado em dois vetores até obter n vetores de um único elemento.

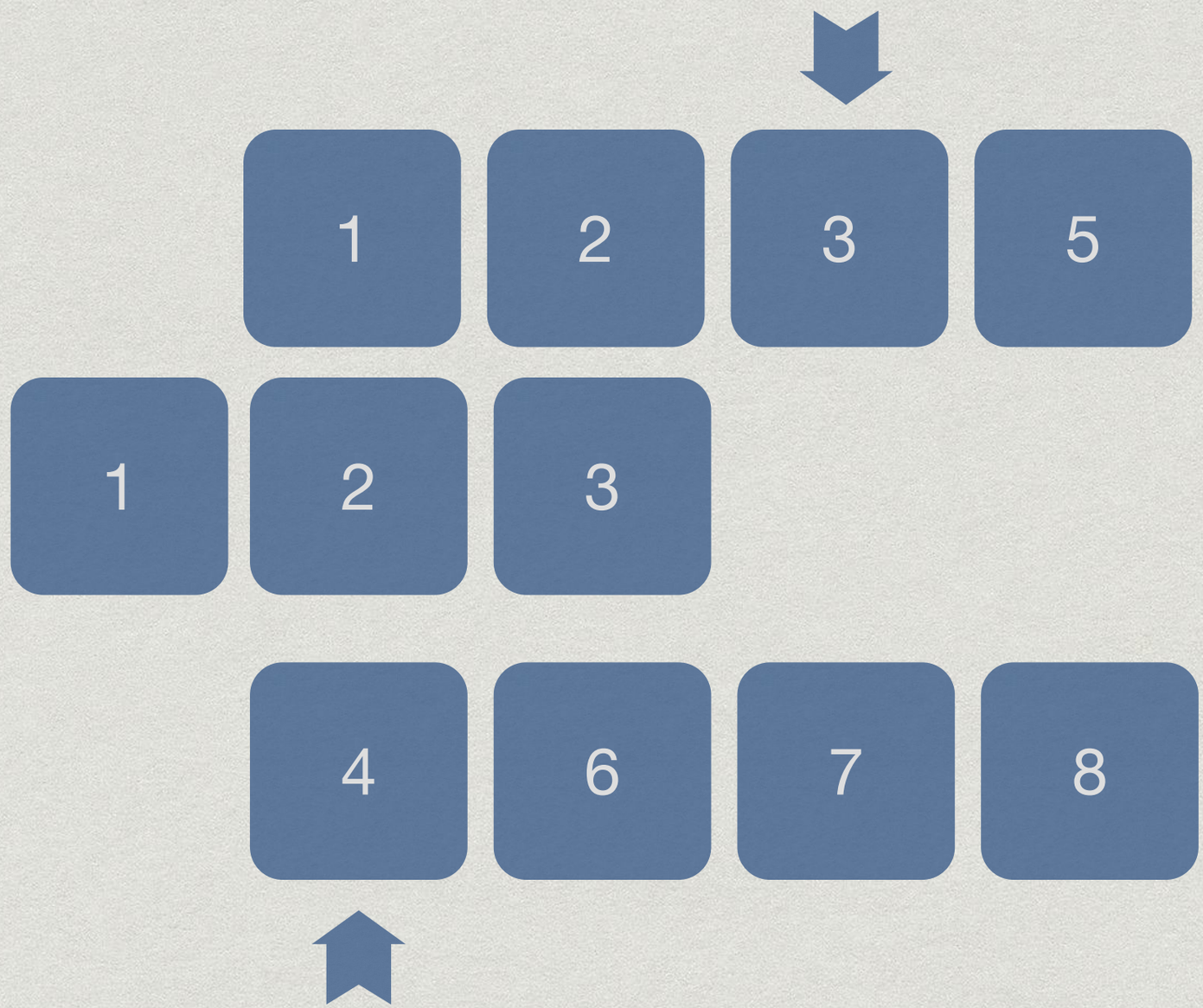
- Aplicar o algoritmo de merge tendo como entrada 2 vetores de um elemento e formando um vetor ordenado de dois elementos.

- Repetir este processo formando vetores ordenados cada vez maiores até que todo o vetor esteja ordenado.

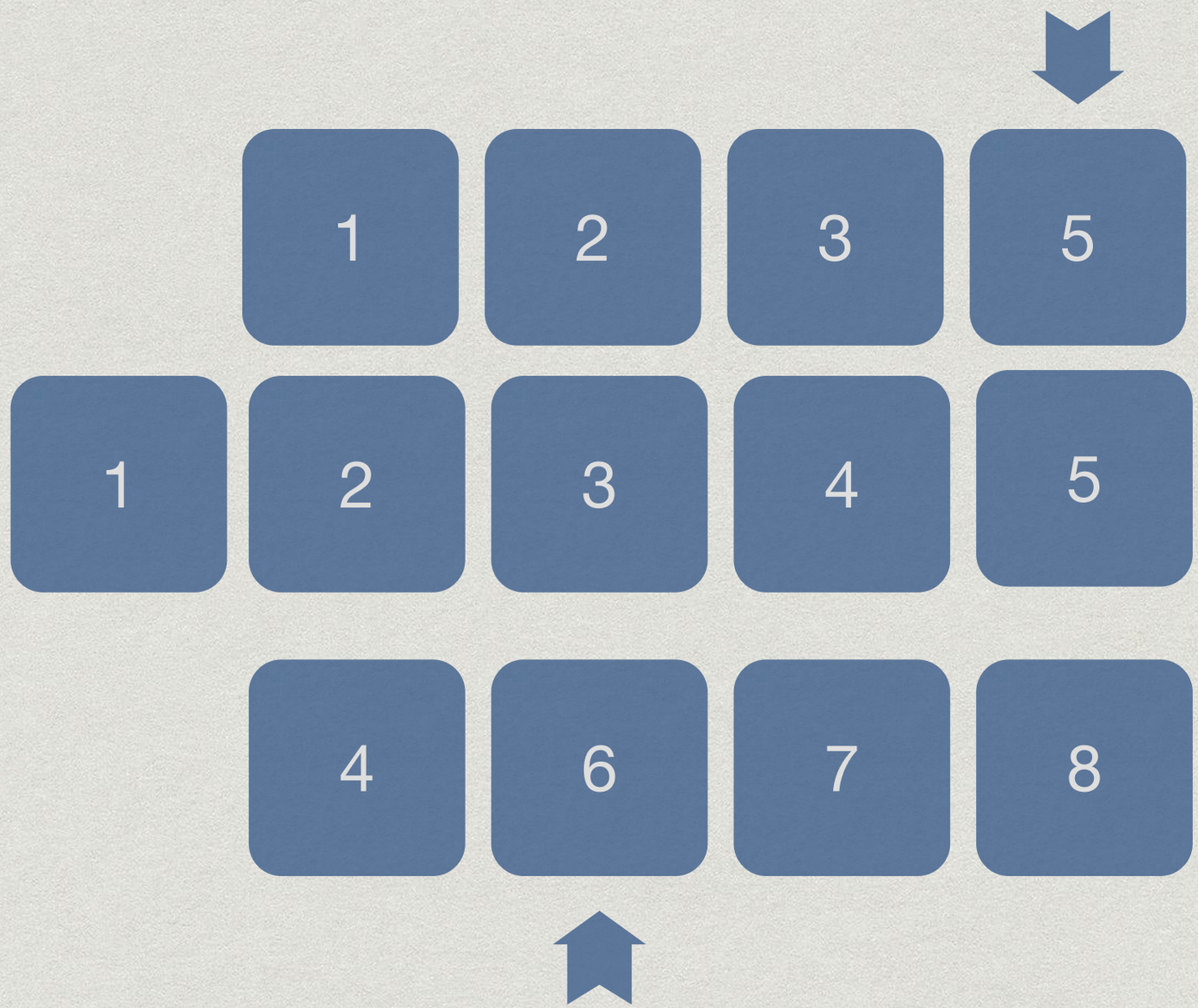

4
1
2
6
4
1
2
6
4
1
2
6
1
4
2
6
1
2
4
6

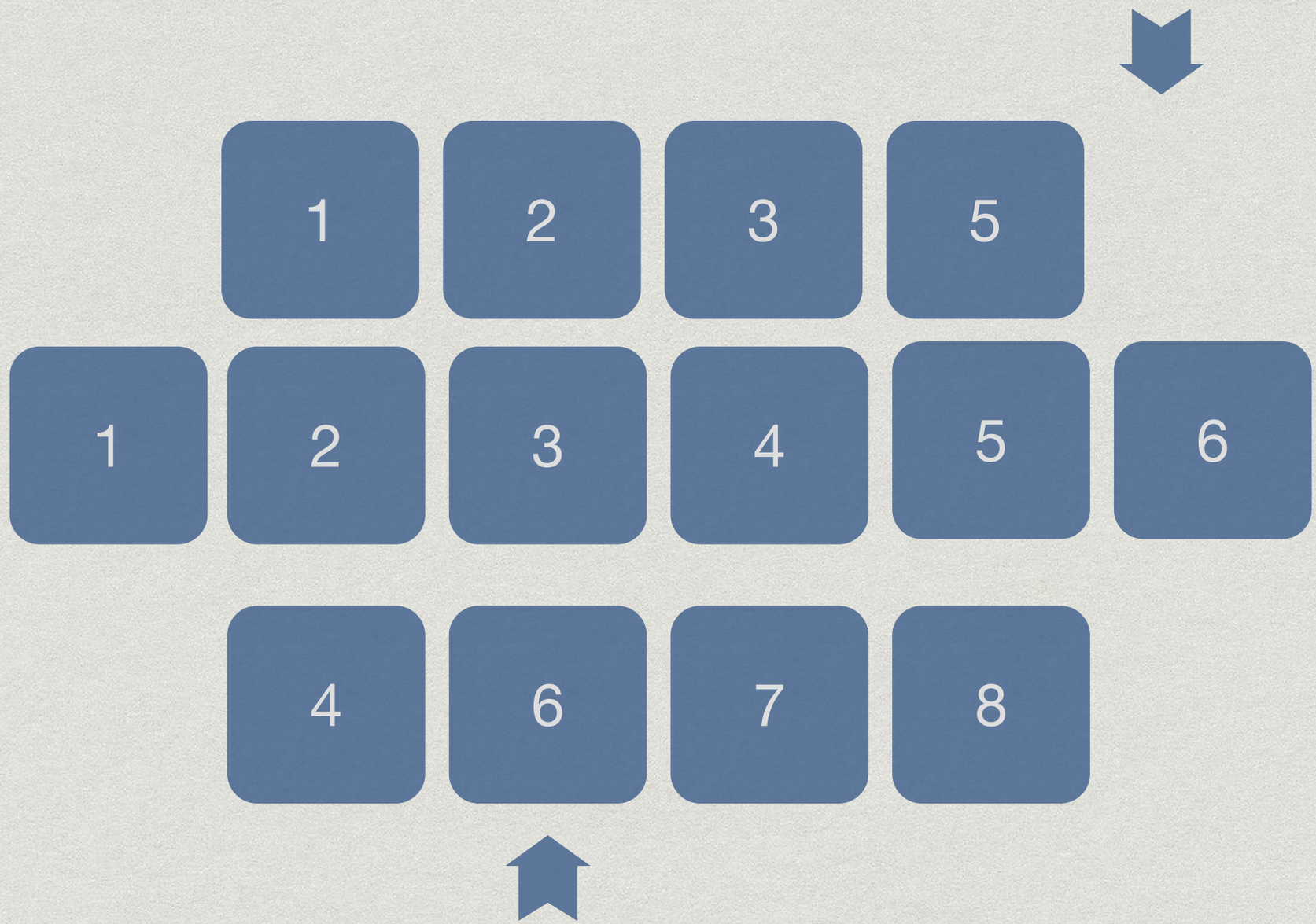

1
2
3
5
1
4
6
7
8

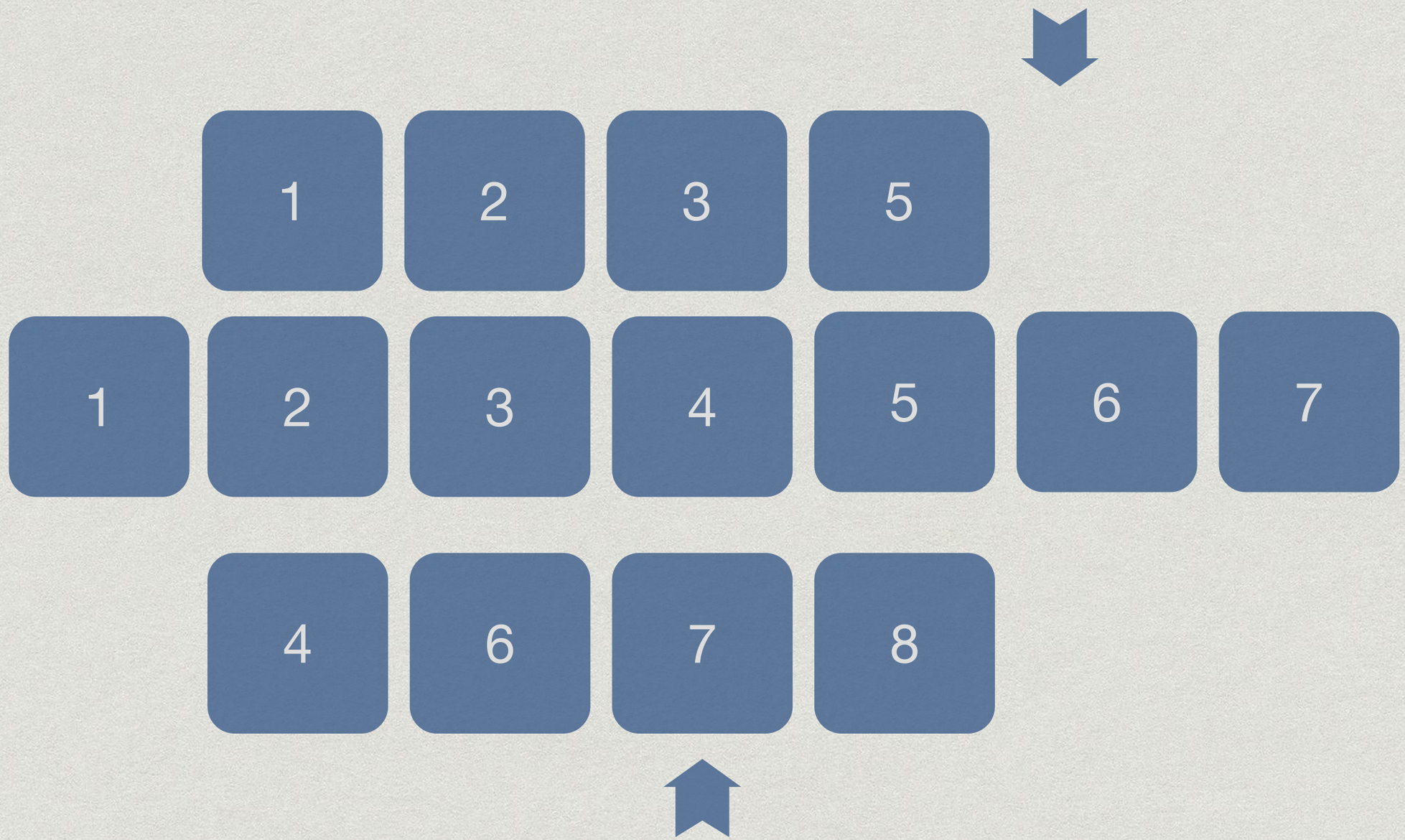

1
2
3
5
1
2
4
6
7
8

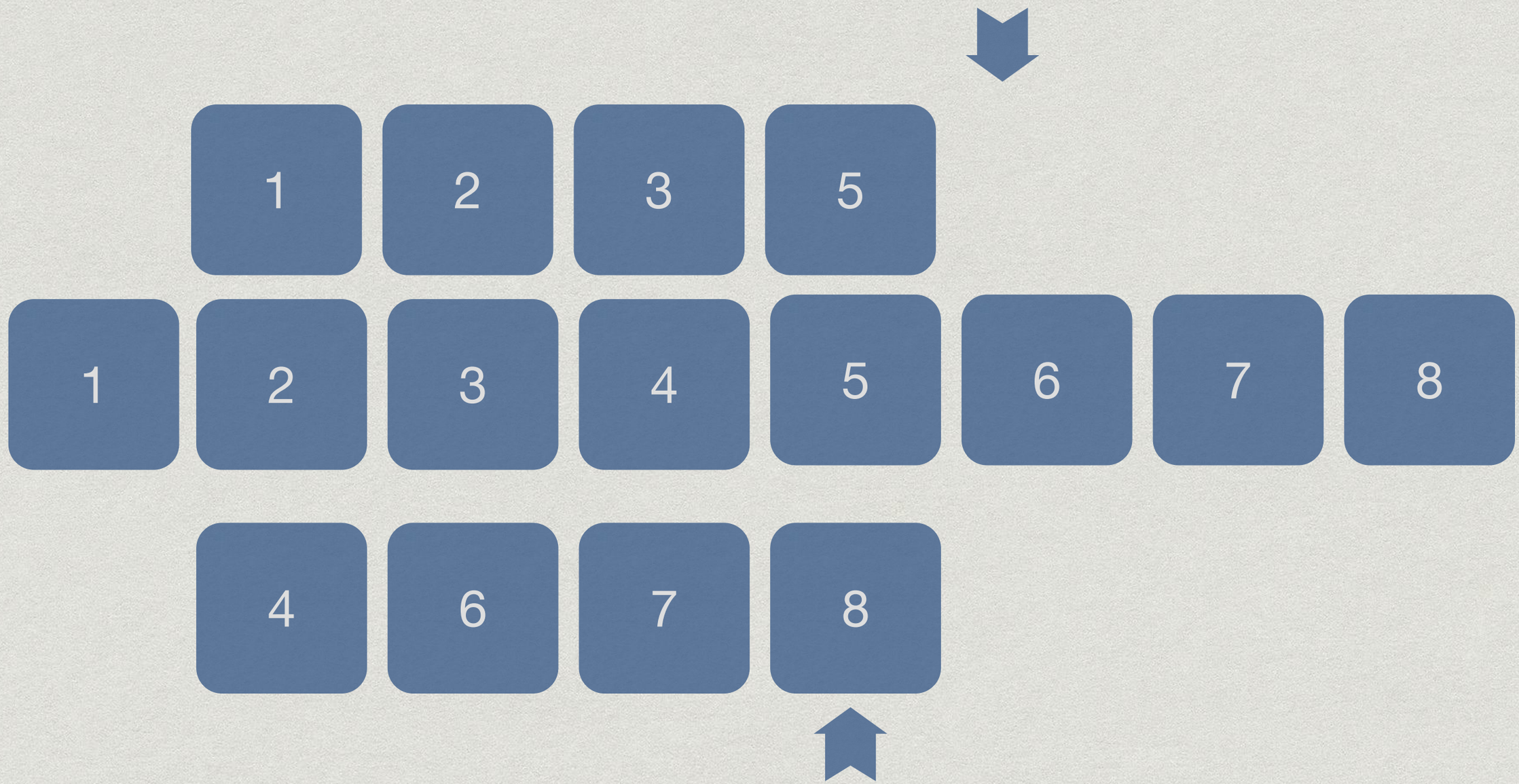

1
2
3
5
1
2
3
4
6
7
8

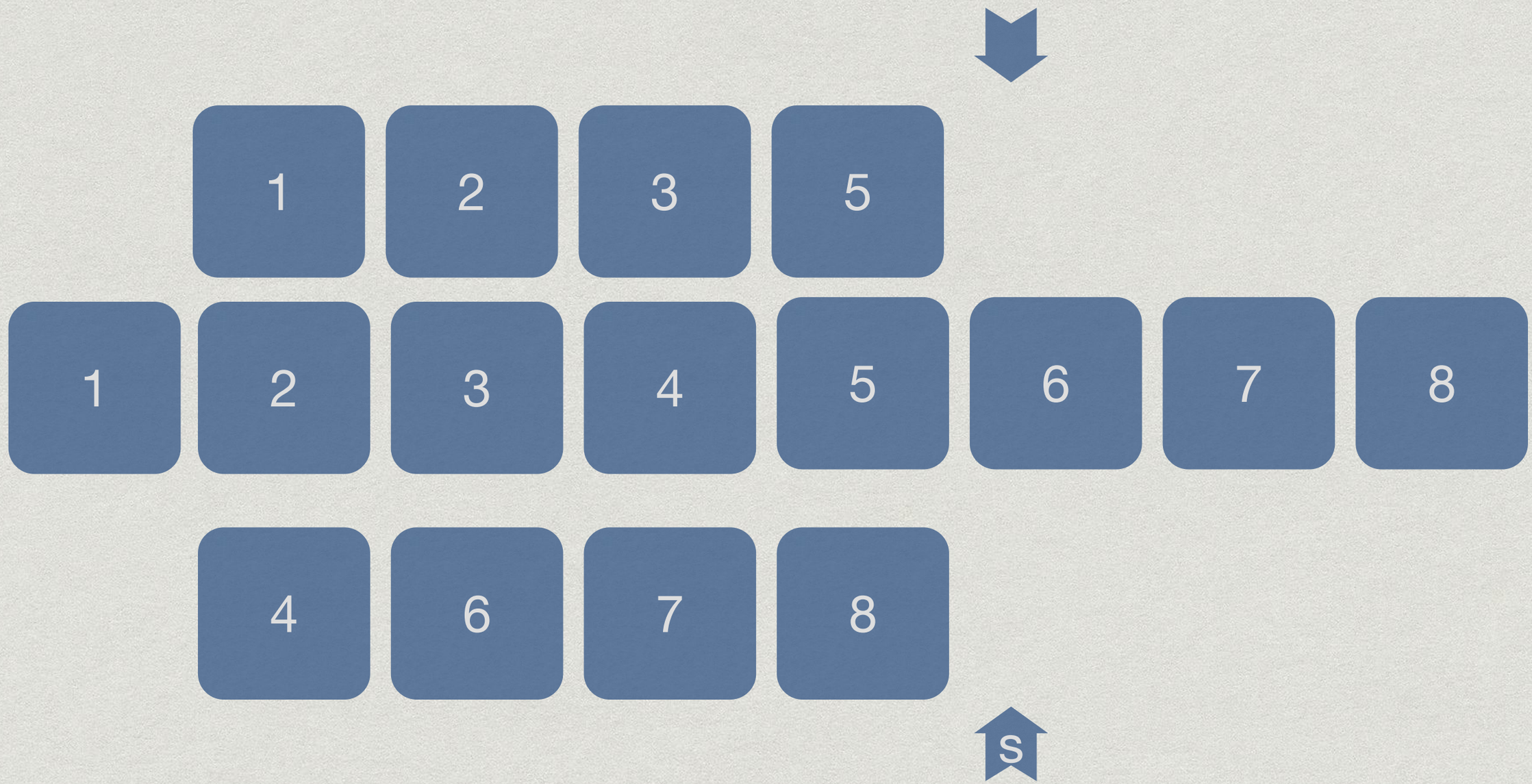
1
2
3
5
1
2
3
4
4
6
7
8


1
2
3
5
1
2
3
4
5
4
6
7
8


1
2
3
5
1
2
3
4
5
6
4
6
7
8


1
2
3
5
1
2
3
4
5
6
7
4
6
7
8


1
2
3
5
1
2
3
4
5
6
7
8
4
6
7
8


1
2
3
5
1
2
3
4
5
6
7
8
4
6
7
8
s

- **Método Mergesort - Exemplo**

Vetor inicial: (23 17 8 15 9 12 19 7)

Passo 1 : (23 17 8 15 | 9 12 19 7)

Passo 2 : (23 17 | 8 15 | 9 12 7 19)

Passo 3 : (23 | 17 | 8 15 | 9 12 7 19)

Passo 4 : (17 23 | 8 15 | 9 12 7 19)

Passo 5 : (17 23 | 8 | 15 | 9 12 7 19)

Passo 6 : (17 23 | 8 15 | 9 12 7 19)

Passo 7 : (8 15 17 23 | 9 12 7 19)

Passo 8 : (8 15 17 23 | 9 12 | 7 19)

Passo 9 : (8 15 17 23 | 9 | 12 | 7 19)

Passo 10 : (8 15 17 23 | 9 12 | 7 19)

Passo 11 : (8 15 17 23 | 9 12 | 7 | 19)

Passo 12 : (8 15 17 23 | 9 12 | 7 19)

Passo 13 : (8 15 17 23 | 7 9 12 19)

Passo 14 : (7 8 9 12 15 17 19 23)

Vetor final: (7 8 9 12 15 17 19 23)

QUICK SORT

ORDENAÇÃO III

Plano Básico

- Embaralhe o vetor - Opcional
  - Melhora o desempenho
- Particiona de forma que alguns j
  - entrada a [j] está no lugar correto
  - nenhuma entrada maior que a[j] vai estar à esquerda de j
  - nenhuma entrada menor que a[j] vai estar à direita de j
- Ordena cada sub-vetor recursivamente.

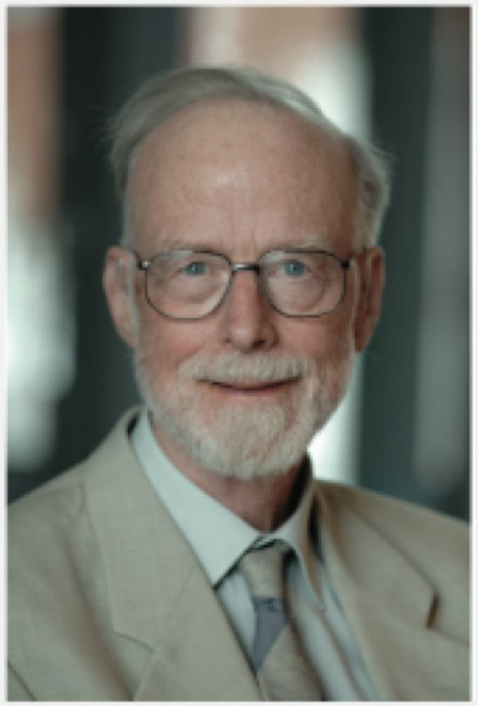
input Q U I C K S O R T E X A M P L E
shuffle K R A T E L E P U I M Q C X O S
partitioning item
partition E C A I E K L P U T M Q R X O S
not greater
not less
sort left A C E E I K L P U T M Q R X O S
sort right A C E E I K L M O P Q R S T U X
result A C E E I K L M O P Q R S T U X

Exemplo

Autor

“Existem duas maneiras de construir um software: uma maneira é torná-lo tão simples que obviamente não há deficiências, e a outra maneira é torná-lo tão complicado que não há deficiências óbvias. O primeiro método é muito mais difícil. "

- Tony Hoare
- Inventou o quick sort para traduzir Russo para Inglês
  - Apesar de não saber como implementá-lo
- Implementado com Algol 60

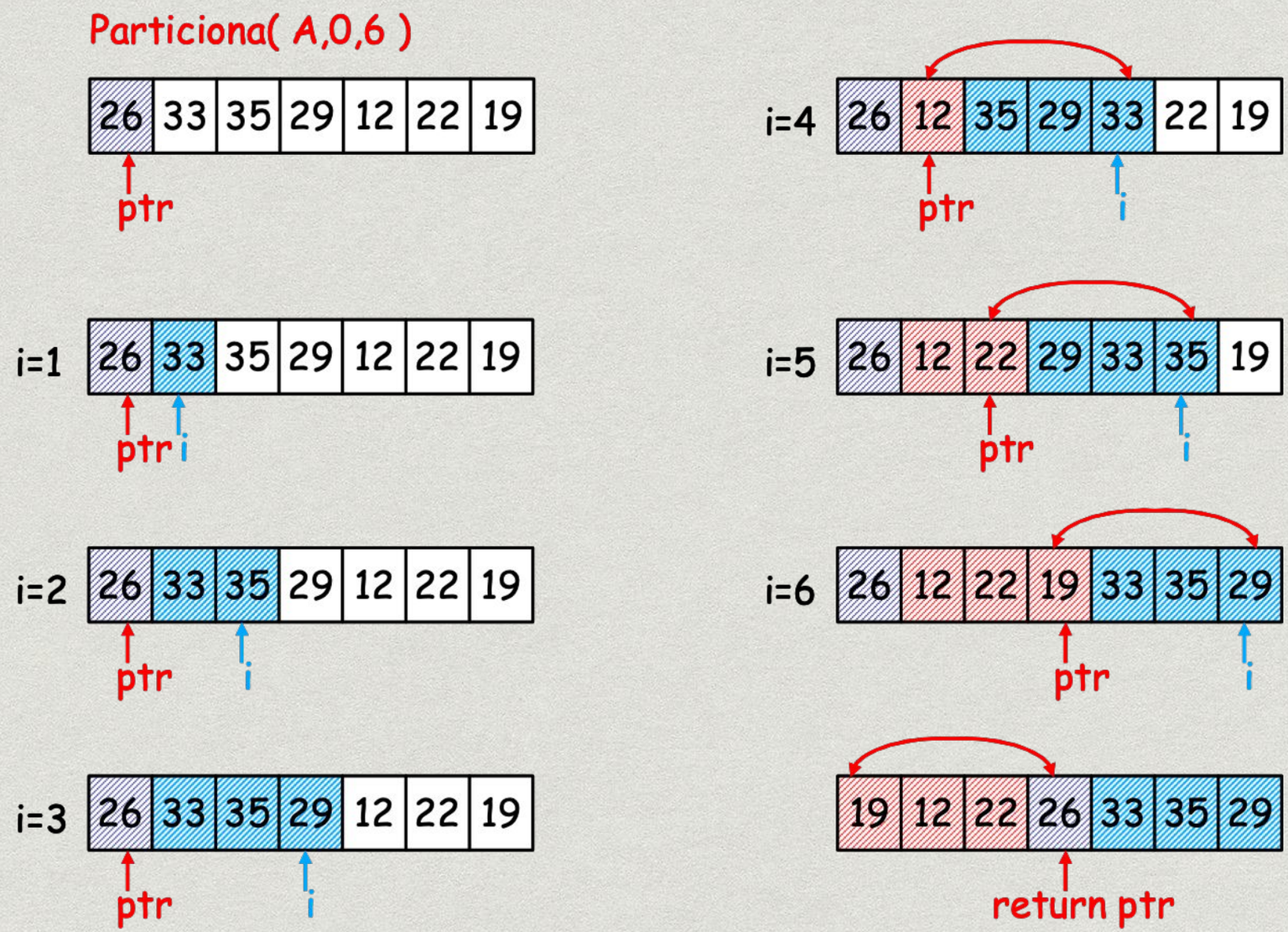


Tony Hoare

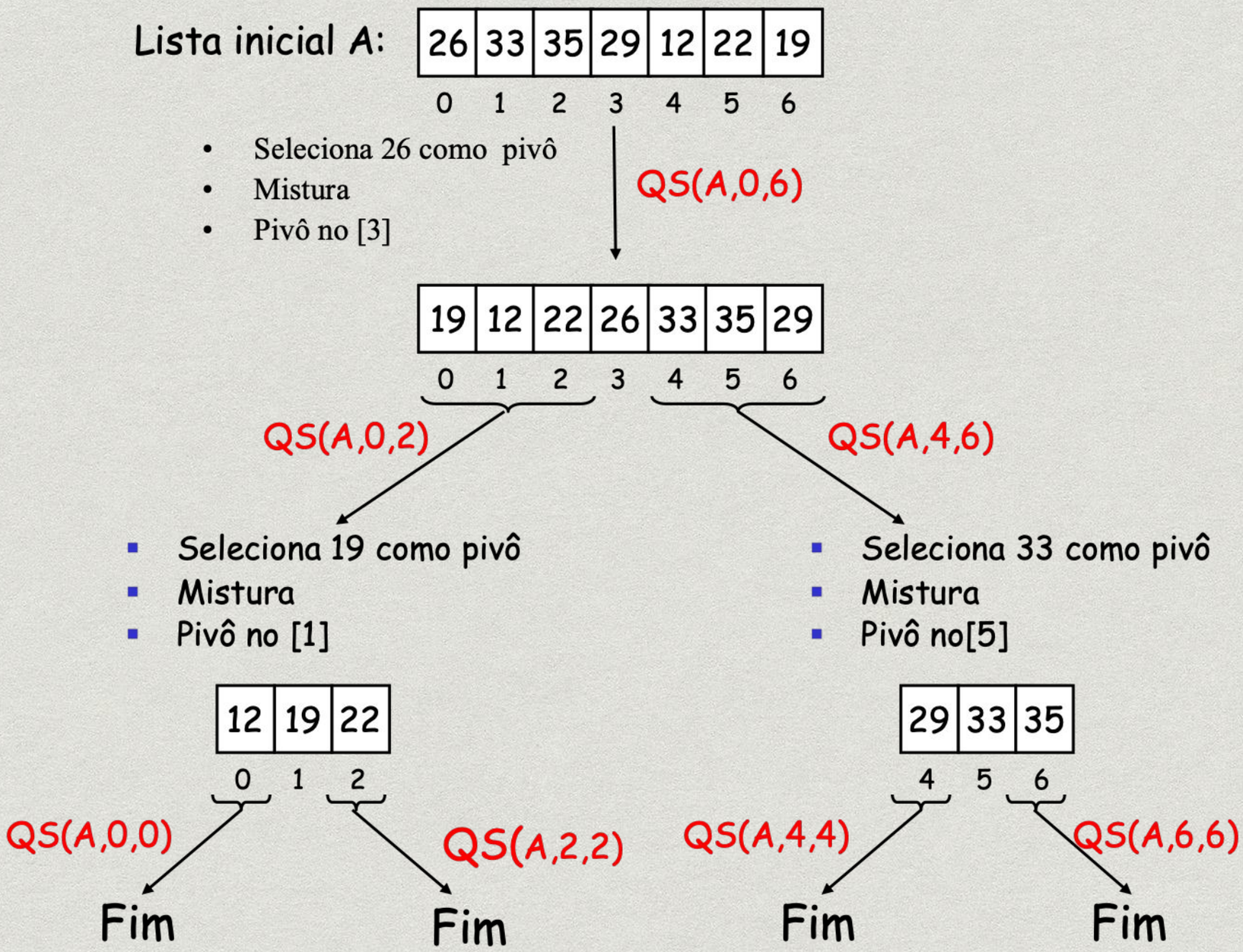
Particiona

- Repita até os ponteiros i e j se cruzarem.
  - Examine i da esquerda para a direita, desde que (a[i] < a[lo]).
  - Examine j da direita para a esquerda, desde que (a[j]> a[lo]).
  - Troque a[i] com a[j].
- Quando os ponteiros se cruzam.
  - Troque a[lo] com a[j].

Execução Particiona



Execução Geral



Discussão

- Comparado com os demais métodos é o que apresenta, em média, o menor tempo de classificação.
- Tem um desempenho logarítmico - O(n log2n)
  - É o que apresenta o menor número de operações elementares.
- As razões porque quicksort é mais eficiente
  - É que o problema de divisão é executado no mesmo vetor e muito eficientemente
  - Não precisa copiar o vetor ordenado
- Para vetores muito pequenos (N<=20) quicksort não é tão eficiente quanto o algoritmo de inserção, pois como ele é recursivo seria chamado várias vezes diminuindo sua eficiência.
- Não é estável!

Análise Empírica

- Estimativas de tempo de execução:
  - O PC doméstico executa 10^8 comparações / segundo.
  - Supercomputador executa 10^{12} comparações / segundo

	insertion sort (N^2)			mergesort (N log N)			quicksort (N log N)		
computer	thousand	million	billion	thousand	million	billion	thousand	million	billion
home	instant	2.8 hours	317 years	instant	1 second	18 min	instant	0.6 sec	12 min
super	instant	1 second	1 week	instant	instant	instant	instant	instant	instant

Lição 1: Bons algoritmos são melhores que supercomputadores

Lição 2: Ótimos algoritmos são melhores que bons algoritmos

PRÓXIMA AULA

- Prática